

## 🎓 Day 12: Advanced SQL + Security + OOP (6h)

### 🔒 APIs + MySQL + OOP Protection

#### Slide 1: Overview – Why Advanced SQL?

- ✓ إضافة بيانات - POST ✓ جلب البيانات - GET ✓ (CRUD) تقوم بالعمليات الأربعة APIs حتى الآن أنشأنا
- حذف - DELETE ✓ تعديل - PUT

اليوم سنتعمق في:

- Joins
- Subqueries
- GROUP BY, HAVING, ORDER BY, LIMIT
- CREATE VIEW
- SQL Injection
- XSS Prevention
- API Security Best Practices
- APIs في بناء OOP استخدام
- أنواع إضافية من الهجمات: Exploits, MITM, Protocol Attacks

باستخدام كائنات `students`, `courses`, `enrollments`, `admin` \ المشروع المرتبط  تستخدم جداول APIs \ OOP (`Student`, `Course`, `Enrollment`).

#### Slide 2: CRUD with API & SQL Mapping

| HTTP Method | SQL Equivalent | مثال تطبيقي     |
|-------------|----------------|-----------------|
| GET         | SELECT         | get.php?id=3    |
| POST        | INSERT         | post.php        |
| PUT         | UPDATE         | put.php?id=2    |
| DELETE      | DELETE         | delete.php?id=7 |

✓ (OOP) مثال عملي:

```
require_once "Course.php";  
$course = new Course($conn);
```

```
$data = $course->getById($_GET['id']);  
echo json_encode($data);
```

## Slide 2.5: MySQL (OOP) إعداد الاتصال بـ

```
class Database {  
    private $host = "localhost";  
    private $username = "root";  
    private $password = "";  
    private $dbname = "school";  
    private $port = 3306;  
    public $conn;  
  
    public function connect() {  
        $this->conn = new mysqli(  
            $this->host, $this->username,  
            $this->password, $this->dbname,  
            $this->port  
        );  
        if ($this->conn->connect_error) {  
            die("Connection failed: " . $this->conn->connect_error);  
        }  
        return $this->conn;  
    }  
}
```

## Slide 2.6: Trait استخدام للاتصال

```
trait MySQLConnection {  
    private $host = "localhost";  
    private $username = "root";  
    private $password = "";  
    private $dbname = "school";  
    private $port = 3306;  
    public $conn;  
  
    public function connect() {  
        $this->conn = new mysqli(  
            $this->host, $this->username,  
            $this->password, $this->dbname,  
            $this->port  
        );  
    }  
}
```

```

    );
    if ($this->conn->connect_error) {
        die("Connection failed: " . $this->conn->connect_error);
    }
    return $this->conn;
}
}

```

ثم:

```

class Database {
    use MySQLConnection;
}

```

## Slide 2.7: مقارنة بين `new mysqli()` و `Database class`

| مقارنة                 | <code>new mysqli()</code> | <code>Database class</code> |
|------------------------|---------------------------|-----------------------------|
| سهولة الاستخدام        | ✓                         | ✗                           |
| إعادة الاستخدام        | ✗                         | ✓                           |
| تنظيم الكود            | ✗                         | ✓                           |
| مناسب للمشاريع الكبيرة | ✗                         | ✓                           |

## Slide 3: SQL JOIN - Why?

```

SELECT s.name, c.title
FROM students s
JOIN enrollments e ON s.id = e.student_id
JOIN courses c ON c.id = e.course_id;

```

6 Task:

```

SELECT c.title, COUNT(e.student_id) AS total
FROM courses c
LEFT JOIN enrollments e ON c.id = e.course_id
GROUP BY c.id;

```

## Slide 3.5: CREATE VIEW – طريقة إنشاء View في SQL

VIEW عبارة عن استعلام محفوظ يمكنك التعامل معه كأنه جدول

```
CREATE VIEW student_courses AS
SELECT s.name AS student_name, c.title AS course_title
FROM students s
JOIN enrollments e ON s.id = e.student_id
JOIN courses c ON c.id = e.course_id;
```

للاستخدام:

```
SELECT * FROM student_courses WHERE course_title = 'Web Development';
```

### 📌 الفرق بين View و Table:

- لا تخزن بيانات فعلية بل تحفظ الاستعلام فقط View
- تُستخدم لتبسيط استعلامات معقدة وإعادة استخدامها
- يمكن استخدامها للقراءة فقط (حسب نوعها)

### 💡 فائدة:

- تقليل التكرار
- APIs تبسيط منطق العرض للمستخدمين أو الـ

## Slide 4: Subqueries – المفهوم والتطبيق

Subquery = استعلام داخل استعلام

مثال 1: طلاب سجلوا في أكثر من كورس

```
SELECT name FROM students
WHERE id IN (
  SELECT student_id FROM enrollments
  GROUP BY student_id
  HAVING COUNT(*) > 1
);
```

## مثال 2: طالب سجل في أكبر عدد كورسات

```
SELECT name FROM students
WHERE id = (
  SELECT student_id FROM enrollments
  GROUP BY student_id
  ORDER BY COUNT(*) DESC
  LIMIT 1
);
```

## مثال 3: الطلاب الذين لم يسجلوا في أي كورس

```
SELECT name FROM students
WHERE id NOT IN (SELECT student_id FROM enrollments);
```

## مع أمثلة إضافية GROUP BY شرح Slide 5

GROUP BY تُستخدم لتجميع النتائج حسب عمود معين

### مثال 1: عدد الكورسات لكل طالب

```
SELECT student_id, COUNT(*) as total
FROM enrollments
GROUP BY student_id;
```

### مثال 2: كم طالب مسجل في كل كورس

```
SELECT course_id, COUNT(*) as total
FROM enrollments
GROUP BY course_id;
```

### ملاحظة:

- GROUP BY يجب أن تُستخدم مع دوال تجميع مثل COUNT, SUM, AVG, MAX, MIN

...